

VIZZA INSURE AI

Complete Project Documentation

AI-Powered Outbound & Web-Based Insurance Voice Assistant

Version: 1.0.0

Target Audience: Business Owner & Engineering Team

Date: May 25, 2026

Confidentiality: Internal Only

Table of Contents

Section 1: Project Overview Page 3

Section 2: Complete Architecture Overview Page 5

Section 3: Backend Details Page 7

Section 4: Frontend Details Page 10

Section 5: Speech to Text (STT) Details Page 12

Section 6: Text to Speech (TTS) Details Page 14

Section 7: AI Brain (Reasoning Engine) Page 16

Section 8: Telephony & Call Flow Details Page 18

Section 9: Database & Schema Details Page 20

Section 10: How to Run the Project (Step-by-Step) Page 22

Section 11: List of API Keys and Services Page 24

Section 12: Known Issues & Technical Limitations Page 25

Section 13: Future System Improvements Page 26

SECTION 1 - PROJECT OVERVIEW

Vizza Insure AI (also referred to as SafeShield Voice Assistant in the source code) is a state-of-the-art, artificial intelligence-powered outbound voice assistant and conversational sales platform. It has been custom-built specifically for Vizza Insurance Company to transform, automate, and scale their customer outreach and sales operations.

At its core, Vizza Insure AI operates as an intelligent sales agent capable of placing real phone calls to customers or conversing with users through a web browser. The system leverages advanced speech-to-text, natural language processing, generative AI, and text-to-speech technologies to deliver human-like, low-latency conversations.

THE BUSINESS PROBLEM IT SOLVES

Traditional insurance calling campaigns rely heavily on human tele-callers. This model faces several severe limitations:

1. **Operational Costs:** Paying hundreds of human callers is expensive. Outbound calling campaigns have high lead-to-connect ratios, meaning agents spend most of their time dialing numbers, hearing busy signals, or handling unanswered calls instead of pitching.
2. **Scalability:** A human team can only make a finite number of calls per hour. Scaling up a campaign for a new policy launch requires hiring and training new staff, which takes weeks.
3. **Quality & Consistency:** Human agents face fatigue, resulting in inconsistent pitch quality, missing follow-ups, and varying adherence to compliance guidelines.
4. **Retention and Burnout:** Tele-calling has one of the highest attrition rates in the corporate world, leading to continuous hiring costs.

Vizza Insure AI solves these problems by providing:

1. **Instant Scalability:** The system can dial thousands of customer phone lines concurrently, performing massive outreach in minutes.
2. **Cost Efficiency:** It reduces the cost per call to a fraction of a human agent's cost, operating 24/7 without fatigue.
3. **Automated Campaign Execution:** Operations teams can upload bulk lists, and the system automatically scrubs DNC (Do Not Call) list numbers, respects calling hours regulations (e.g., 9 AM to 8 PM in India), and dials sequentially.
4. **Consistent Quality & Compliance:** The AI brain strictly adheres to the sales guidelines, maintains a warm and professional tone, never leaves compliance scripts, and logs every call details.

CUSTOMER CALL EXPERIENCE

When a customer receives a call from Vizza Insure AI, they experience a warm, human-like voice:

1. **Opening Greeting:** The call begins with a polite, enthusiastic introduction from one of two configured agents, Priya (female) or Arjun (male). For example: "Hello! I'm Priya from Vizza Insurance. Do you have 2 minutes?"
2. **Active Listening & Low Latency:** The system listens to the customer and responds in under 1.5 seconds. It handles natural speech patterns, pauses, and back-and-forth conversation.
3. **Indian English, Tamil, and Tanglish Support:** The assistant automatically detects whether the customer is speaking English, Tamil, or Tanglish (a blend of English and Tamil, written phonetically) and switches its language

mode instantly. It avoids overly formal Tamil and instead communicates in a friendly, conversational "Tanglish" dialect popular in India.

4. Dynamic Pitching: It gauges customer interest and pitches specific products (Vizza Health Plus, Vizza Drive Safe, Vizza Life Secure, Vizza Home Guard).

5. Objection Handling: If the customer says "I am busy," "I already have insurance," or "I am not interested," the system gracefully responds with warm, non-pressuring discovery questions to salvage the lead.

6. Human Escalation: If the customer asks to speak to a manager or human agent, the AI immediately says: "Of course! Transferring you now," and triggers a call transfer.

OPERATIONS TEAM DASHBOARD

The operations dashboard provides an interface to monitor and control the system:

1. Single Outbound Calling: The team can input a phone number, specify the customer's name, choose the voice agent (Priya/Arjun), and initiate a call.

2. Bulk Campaign Dialer: Team members can paste a list of comma-separated phone numbers, select an agent, and launch a campaign. The backend dials numbers sequentially, enforces a 2-second gap between dials to prevent trunk congestion, and schedules callbacks for unanswered or busy lines.

3. Real-Time Tracking: The dashboard displays live call statistics (active call count) and updates status logs.

4. Analytics Suite: Operations can view aggregate statistics including total calls made, answer rate, average call duration, escalation rate, policy interest distribution (which products are popular), and call status breakdowns.

5. Audit & Logs: The system stores granular call events (e.g. call started, greeting played, customer utterance, AI response sent, call ended) and complete text transcripts of every conversation, allowing easy auditing.

THE TWO SYSTEM MODES

Vizza Insure AI supports two distinct modes of execution, both powered by the exact same AI brain:

1. Web Browser Chat & Voice Mode: Accessible via a web page (browser-call.html). Users can chat with the assistant using text input or talk directly into their microphone. The client-side application handles microphone capture, uses the browser Web Speech API for speech-to-text, and sends the text to the backend. The backend generates the AI response and returns neural TTS audio, which the browser plays. This mode is excellent for local testing and website support.

2. Real Phone Call Mode: This mode places real calls to landlines and mobile phones. It uses Twilio for telephony, establishing a WebSocket stream between Twilio and the backend server. The backend receives raw 8kHz mono mu-law audio from the customer's microphone, runs silence detection, transcribes speech using Groq's Whisper API, processes the conversation using the AI brain, synthesizes neural TTS, transcodes the audio to telephony-format mu-law via FFmpeg, and streams it back to the customer.

HOW BOTH MODES USE THE SAME AI BRAIN

Both the browser voice mode and the phone call mode communicate with the same core backend server. When a customer speaks in either mode, the voice is transcribed into text. This text is appended to the conversation history, which is maintained in isolated sessions.

The backend then runs its central language detection and dynamic system prompt builder. It sends the formatted conversation history to the primary AI engine (Groq's Llama 3.3 model). The brain generates a textual response, which is fed to the respective speech-to-text and text-to-speech pipelines. This ensures that the assistant's personality, knowledge base, policies, and behavior remain identical regardless of whether a customer is chatting

on a website or talking on a mobile phone.

SECTION 2 - COMPLETE ARCHITECTURE OVERVIEW

The system architecture of Vizza Insure AI is designed as a modular, layered stack that handles real-time audio capture, processing, AI generation, and database logging.

SYSTEM LAYERS

1. Customer Interface Layer:

- Web Client: The browser interface (browser-call.html) using HTML5, CSS (Dark Apple Glassmorphism styling), and JavaScript. It utilizes the MediaRecorder API for audio capture and the browser Web Speech API for on-device speech-to-text.

- Mobile / Landline Phone: The customer's physical telephone. Audio is captured and played back natively through standard cellular networks.

2. Telephony Layer (Twilio Gateway):

- Twilio acts as the bridge between the Public Switched Telephone Network (PSTN) and the backend server. It handles call routing, dialing, call state updates, and establishes WebSocket media streams.

- When a call is answered, Twilio executes TwiML instructions returned by the server, opening a bi-directional WebSocket connection to stream raw audio.

3. Speech Processing Layer:

- Speech-to-Text (STT):

- * Browser Mode: Uses the browser's built-in Web Speech API (webkitSpeechRecognition) for local, fast speech-to-text.

- * Telephony Mode: Streams raw mu-law audio over WebSocket. The server accumulates audio, wraps it in a WAV header, and POSTs it to Groq's high-speed Whisper API (whisper-large-v3-turbo).

- Text-to-Speech (TTS):

- * Browser Mode: Calls the /api/tts route which uses Microsoft Edge Neural TTS (msedge-tts) to generate high-fidelity Indian English/Tamil MP3 streams. Falls back to browser speechSynthesis if needed.

- * Telephony Mode: Utilizes Microsoft Edge Neural TTS. Since Twilio requires 8kHz mono mu-law audio, the server transcodes the generated high-quality MP3 stream into telephony-standard mu-law format in real-time using FFmpeg (fluent-ffmpeg).

4. AI Brain Layer (Conversation Engine):

- Uses Groq Cloud API's llama-3.3-70b-versatile model as the core reasoning engine.

- Receives the conversation history and the dynamic system prompt.

- Executes dynamic language detection and matches system prompts to three sub-modes: English, Tanglish, and Tamil Script.

5. Backend Server Layer (Fastify Core):

- A high-performance Node.js server built using the Fastify framework.

- Manages WebSocket endpoints (/browser-stream and /api/calls/media-stream), coordinates the STT -> AI -> TTS pipeline, handles session management, exposes REST API routes, and serves the static frontend interface.

6. Database Layer (Persistence):

- Employs Prisma ORM to interact with a SQLite database (prisma/dev.db).

- Stores customer lists, call campaigns, call logs, granular call events, and generated sales leads.

DATA FLOW STEPS (VOICE SPEECH TO AI RESPONSE)

Here is the step-by-step journey of data when a customer speaks on a phone call until the voice agent responds:

Step 1: Customer Speaks

The customer speaks into their phone microphone. The phone captures the sound waves and transmits them over the cellular network to Twilio.

Step 2: Audio Streaming

Twilio digitizes the audio into 8kHz mono mu-law format, base64-encodes it, and sends it over a bi-directional WebSocket connection to the Fastify server's `/api/calls/media-stream` endpoint.

Step 3: Silence and Energy Monitoring

The backend WebSocket handler receives the base64 chunks, decodes them into raw binary buffers, and feeds them into the Speech-to-Text stream (STTStream). It checks the RMS energy of each audio chunk to filter background noise (using `getMulawEnergy`). If the user stops speaking for more than 1.5 seconds, the silence timer triggers the processing of the accumulated buffer.

Step 4: Speech-to-Text Transcription

The server wraps the accumulated raw mu-law audio buffer in a WAV header (`wrapMulawInWav` utility). It then sends this buffer as an audio file (`audio.wav`) to the Groq Whisper API (`whisper-large-v3-turbo`). Groq transcribes the audio and returns the plain text transcript.

Step 5: Session Logging and Event Capture

The transcript is appended to the call's conversation history in the `sessionManager`. Simultaneously, a `CallEvent` of type `utterance_received` is asynchronously logged in the SQLite database containing the transcribed text.

Step 6: AI Brain Inference

The server detects the language of the transcript (English vs. Tamil/Tanglish). It constructs a dynamic system prompt based on the agent's name (Priya/Arjun) and detected language. The history and system prompt are sent to Groq Llama 3.3. Groq returns a concise, conversational text response.

Step 7: Background Intent Analysis

While the response is being prepared, the server runs an asynchronous intent analysis (using `analyzeIntent`) on the customer's transcript to check if they want to buy, have objections, or requested a callback. This data is recorded in the customer's session state and a Lead is created/updated in the database if necessary.

Step 8: Text-to-Speech Synthesis

The AI's text response is sent to the TTS service. The service uses `msedge-tts` to generate a high-quality neural voice MP3 file (e.g. using Priya's Indian English voice `en-IN-NeerjaNeural`).

Step 9: Audio Transcoding

The generated MP3 stream is transcoded in real-time using `FFmpeg` into 8kHz mono mu-law format to match Twilio's telephony requirements.

Step 10: Playback to Customer

The transcoded mu-law buffer is split into 160-byte chunks (representing 20ms of audio each) to ensure smooth playback. The server base64-encodes these chunks and sends them over the WebSocket connection to Twilio.

Twilio receives the chunks and streams the voice back to the customer's phone, completing the loop.

SECTION 3 - BACKEND

The backend of Vizza Insure AI is a high-performance, asynchronous server designed to handle real-time WebSockets, streaming audio transcoding, database CRUD operations, and external API requests.

PROGRAMMING LANGUAGE: TYPESCRIPT VS JAVASCRIPT

The backend is written in TypeScript.

- Why TypeScript?

JavaScript is dynamically typed, meaning variable types are determined at runtime. This can lead to subtle bugs (e.g., trying to read properties from an undefined object) that are only discovered during active phone calls.

TypeScript introduces static typing, interfaces, and compile-time error checking. It acts as a layer on top of JavaScript. During development, the TypeScript compiler checks the code for type mismatches, syntax errors, and missing parameters. It provides rich IDE autocomplete, making refactoring safe.

- Choice for Vizza: The choice of TypeScript was crucial because the system manages complex asynchronous lifecycles (WebSockets, database events, timers, audio buffering). Static typing ensures that state transitions, environment schemas (validated via Zod), and data payloads are rigorously structured, preventing runtime crashes.

SERVER FRAMEWORK: FASTIFY VS EXPRESS

The backend server uses the Fastify framework.

- What is Fastify?

Fastify is a web framework for Node.js highly focused on providing the best developer experience with minimal overhead and powerful plugin architecture.

- Why chosen over Express?

1. Performance: Fastify is up to 2-3 times faster than Express, capable of serving significantly more requests per second.
2. Built-in Schema Validation: It has native integration with JSON Schema, allowing request and response payloads to be validated automatically, which speeds up processing and increases security.
3. Structured Plugin Architecture: Fastify uses a clean, encapsulation-friendly plugin system (fastify-plugin). All routing, database connections, and middleware are registered as plugins, avoiding global state pollution.
4. Native WebSocket Integration: Through @fastify/websocket, Fastify handles WebSockets seamlessly within its route architecture, which is essential for low-latency streaming of microphone and telephony audio.

MAIN ENTRY POINT: src/index.ts

The file src/index.ts serves as the application's entry point. When the server starts up, this file performs the following operations in order:

1. Loads Environment Variables: Calls dotenv.config() and schema-validates the variables using Zod (src/config/env.ts).
2. Instantiates Fastify: Creates the Fastify server instance, disabling default logger logs and setting a body limit of 10MB to accommodate CSV uploads.
3. Registers Plugins: Registers CORS (allowing cross-origin requests from the browser), Multipart (for parsing uploaded files), WebSocket, and Formbody (for parsing URLencoded Twilio payloads).
4. Configures Error Handling: Sets the global error handler middleware to catch validation or database errors and

return clean JSON responses.

5. Registers Health Check: Sets up the /health route to verify database, Twilio, Deepgram, and Groq statuses.
6. Serves Static Files: Registers @fastify/static to serve static frontend files (e.g. browser-call.html) directly from the /public folder.
7. Registers API Routes: Mounts routing modules to their respective API prefixes.
8. Registers WebSocket Streams: Binds WebSocket endpoints to stream handlers.
9. Connects to Database: Calls connectDatabase() to establish a connection using Prisma.
10. Binds to Port: Listens on the configured port and host, printing a startup summary console block showing active services and warning about missing API keys.
11. Graceful Shutdown: Listens for SIGTERM and SIGINT signals to close the server and disconnect the database cleanly.

COMPLETE ROUTE FILE REGISTRY

All routes are registered inside modular files in src/routes/:

1. File: src/routes/call.routes.ts (Prefix: /api/call)

Coordinative route handler for Twilio telephony integration.

- POST /api/call/start

Initiates an outbound phone call. Validates body (customerPhone, customerName, agentName), formats phone numbers to E.164, calls the Twilio API to dial the customer, initializes an in-memory session, and returns the Call SID.

- POST /api/call/campaign

Starts a bulk sequential calling campaign. Receives an array of customers, verifies DNC and calling hours, and invokes the campaign background dialer.

- POST/GET /api/call/answer

Twilio Answer Webhook. Triggered when the customer answers their phone. Generates TwiML connecting the call to the media stream WebSocket. Returns a fallback TwiML greeting with <Say> and <Gather> if streaming fails.

- POST /api/call/respond

Twilio Response Webhook (Fallback loop). Executed when the customer speaks in fallback webhook mode. Transcribes voice via Twilio, sends text to Groq, and generates TwiML <Say> and <Gather> responses.

- POST /api/call/status

Twilio Status Webhook. Triggered when a call ends or fails. Cleans up in-memory call sessions.

- GET /api/call/twilio-test

Diagnostic endpoint. Fetches Twilio account details to verify credentials.

- GET /api/call/twiml-test

Diagnostic endpoint. Returns a static TwiML speech response to test webhook routing.

2. File: src/routes/customer.routes.ts (Prefix: /customers)

Manages customer profiles and registrations.

- POST /customers/

Creates a single customer. Checks for duplicate phone numbers and saves the profile. (Protected by authMiddleware).

- POST /customers/upload

Accepts a CSV file upload. Parses records (name, phone, email, preferred calling hours), validates schemas, and upserts them into the database. (Protected by authMiddleware).

- GET /customers/

Lists all customers. Supports page, limit, search, and DNC filters. (Protected by authMiddleware).

3. File: src/routes/dashboard.routes.ts (Prefix: /dashboard)

Exposes reporting and analytics data.

- GET /dashboard/calls

Returns paginated list of call logs, durations, and escalation flags. (Protected by authMiddleware).

- GET /dashboard/calls/:callId

Retrieves a single call log complete with granular events and transcripts. (Protected by authMiddleware).

- GET /dashboard/analytics

Aggregates stats (total calls, answer rate, average duration, escalation rate, policy interest counts). (Protected by authMiddleware).

- GET /dashboard/live

Server-Sent Events (SSE) stream. Pushes live updates of active calls to the dashboard client. (Protected by authMiddleware).

4. File: src/routes/lead.routes.ts (Prefix: /leads)

Sales opportunity registration.

- POST /leads/

Creates or updates a lead. Links leads to Call Logs and Customers using the phone number. (Protected by authMiddleware).

- GET /leads/

Lists leads with status, interested policy, and search filters. (Protected by authMiddleware).

- PATCH /leads/:id

Updates a lead's status (e.g. CONTACTED, CONVERTED) and notes. (Protected by authMiddleware).

5. File: src/routes/test.routes.ts (Prefix: /test)

- POST /test/chat

Proxy chat router. Accepts messages and forwards them to Groq AI. Prevents CORS errors during direct browser testing.

6. File: src/routes/tts.routes.ts (Prefix: /api)

- POST /api/tts

Neural TTS generator. Accepts text and agent parameters, cleans text, synthesizes audio using Microsoft Edge TTS, and returns a raw MP3 buffer.

PORT & HOST CONFIGURATION

The backend server runs on Port 3000 by default. It binds to host 0.0.0.0, allowing it to accept external connections (crucial for ngrok tunnels). This is configured in .env via PORT and HOST variables and loaded via src/config/env.ts.

SECTION 4 - FRONTEND

The frontend of Vizza Insure AI is designed as a single-page web app built to provide a clean, premium, and highly responsive testing ground for the AI.

FRONTEND FILE REGISTRY

The entire frontend is contained within one file:

- Location: public/browser-call.html
- Purpose: Serves the web-based call client and outbound calling dashboard. It contains all HTML structure, inline CSS styles, and client-side JavaScript. This unified structure prevents resource loading errors and ensures the client starts immediately.

HOW THE USER ACCESSES THE FRONTEND

The user opens a web browser and navigates to:

- Local URL: `http://localhost:3000/browser-call.html`
- External Tunnel URL: `https://<your-ngrok-subdomain>.ngrok-free.dev/browser-call.html` (used during Twilio testing so the browser page can load over secure HTTPS).

BROWSER CALL PAGE - STEP-BY-STEP INTERACTIONS

The webpage includes several premium interactive components:

1. Start Call Button:

- When clicked, it triggers browser audio unlocking (`unlockAudioContext()`) to bypass autoplay restrictions.
- Requests microphone permission using `navigator.mediaDevices.getUserMedia`.
- Sets `callActive = true`, displays the "End Call" button, and shows the FAQ section.
- Appends the initial AI greeting ("Hello! I'm Priya from Vizza Insurance...") to the chat window, calls the TTS endpoint to play the audio, and starts client-side speech recognition.

2. End Call Button:

- Sets `callActive = false`.
- Stops speech recognition and cancels any ongoing speech synthesis.
- Resets the avatar to the idle state and clears active timers.

3. Voice Toggle (Priya / Arjun):

- Switches the active agent name. Priya is the female agent; Arjun is the male agent.
- Toggling immediately updates the page labels, plays a short voice introduction (e.g. "Hi, I'm Arjun. How can I help you today?"), and sets the name variable passed to the AI brain.

4. Language Toggle (English / Tamil):

- Changes the speech recognition language code.
- English mode sets recognition to `en-IN` (capable of transcribing Indian English and Tanglish).
- Tamil mode sets recognition to `ta-IN` (optimized for transcribing pure Tamil script).
- Plays a vocal confirmation (e.g., "Tamil mode ready. Naan Tamil la pesuren!").

5. Playback Speed Slider:

- Allows users to adjust the speed of the voice agent from 0.5x to 2.0x.
- Speed values are sent to the TTS engines to speed up/slow down voice responses.

6. Chat Window:

- A scrolling conversation history panel showing messages in chat bubbles.
- User messages appear in purple bubbles aligned to the right. Agent responses appear in grey bubbles aligned to the left.
- Each bubble contains a capitalized sender header (YOU, PRIYA, or ARJUN) for readability.

7. Live Transcript Box:

- Appears at the bottom of the chat window. Shows real-time interim speech-to-text results (e.g. "You said: What does health...") as the user speaks, providing immediate visual feedback.

8. Quick Questions FAQ Panel:

- An interactive panel containing common questions (e.g., "What does health insurance cover?", "Premium for health plan?").
- Clicking any question automatically starts a call (if not already active) and submits the text query to the AI brain, demonstrating the assistant's expertise.

9. Animated Siri-Style Avatar (Voice Orb):

- A circular fluid element that morphs shape and changes color based on agent state:
 - * Idle State (avatar-idle): A slow, purple-to-blue morphing orb indicating the agent is waiting.
 - * Listening State (avatar-listening): A green morphing orb that scale-pulses, indicating the microphone is active and capturing speech.
 - * Thinking State (avatar-thinking): An orange-pink spinning orb accompanied by bouncing dots, indicating the LLM is generating a response.
 - * Speaking State (avatar-speaking): An energetic, multi-colored fast-morphing orb that expands to 1.15x scale, matching the playback of the synthesized audio.

FRONTEND-BACKEND COMMUNICATION CHANNELS

The frontend uses two channels to communicate with the Fastify server:

1. HTTP Fetch Requests (REST API):

- POST /test/chat: Sends the conversation history, agent name, and language. Used to get textual AI replies in the browser.
- POST /api/tts: Sends text and receives a neural voice MP3 audio blob.
- POST /api/call/start: Sends phone number, name, and agent parameters to trigger Twilio outbound phone calls.
- POST /api/call/campaign: Sends bulk numbers list to start campaigns.

2. WebSocket Connections:

- While the HTML client uses fetch requests for simple browser-based testing, the backend also exposes a /browser-stream WebSocket endpoint. When testing streaming mode, raw WebM microphone audio chunks are streamed to the backend, and synthesized audio chunks are streamed back to the browser client, simulating the telephony pipeline.

SECTION 5 - SPEECH TO TEXT

Speech-to-Text (STT) is the transcription technology that converts the spoken words of the customer into text that the AI Brain can comprehend.

WEB BROWSER SPEECH-TO-TEXT

The browser call page utilizes the Web Speech API (SpeechRecognition / webkitSpeechRecognition) built natively into modern browsers (Google Chrome, Microsoft Edge).

- How it works:

1. When a call is active and the agent is not speaking, startListening() instantiates SpeechRecognition.
2. The recognition language is set based on the UI toggle (either en-IN or ta-IN).
3. The browser captures microphone audio locally. It uses local/cloud acoustic models to transcribe the audio streams, firing onresult events.
4. Interim results are shown in the transcript box. Once the user stops speaking, the final transcript is captured, recognition is paused, and handleUserInput() is invoked to send the text to the AI.
5. This client-side processing is 100% free, runs instantly, and requires no API keys.

TELEPHONY PHONE CALL SPEECH-TO-TEXT

For phone calls, the system utilizes Groq's high-speed Whisper Cloud API (whisper-large-v3-turbo) integrated in the backend (src/services/stt.service.ts).

- How it works:

1. Twilio streams raw 8kHz mono mu-law audio over WebSocket to /api/calls/media-stream.
2. The server's STTStream class accumulates these audio buffers in memory.
3. A silence detector monitors the stream. If the incoming audio energy (calculated via Root Mean Square in getMulawEnergy) drops below a threshold of 500 for more than 1.5 seconds, it marks the end of an utterance.
4. The server wraps the accumulated raw mu-law audio buffer in a standard 44-byte WAV header (wrapMulawInWav utility). This conversion is necessary because Whisper cannot read raw mu-law data directly. Wrapping it in a WAV header identifies it as a mu-law file, making it readable.
5. The server uses Groq's toFile helper to send this WAV buffer to the Groq Whisper transcription API.
6. Groq transcribes the speech and returns the text, which is emitted via the transcript event.
7. Note: If the WebSocket media stream fails, Twilio falls back to HTTP webhooks using the <Gather> verb. In this fallback mode, Twilio transcribes the speech on its own servers and posts the text result directly to /api/call/respond.

LANGUAGE CODES

- English (and Tanglish): Configured to use en-IN (Indian English). This code is used in browser speech recognition, Twilio webhook <Gather> verbs, and tells Groq Whisper to translate/transcribe Indian-accented speech.
- Tamil: Configured to use ta-IN (Tamil - India). This directs the speech recognizers to capture Tamil characters correctly.

VOICE-TO-TEXT PROCESSING JOURNEY

User speaks -> Microphone captures analog audio -> Converted to digital signal -> (Browser Mode: Browser

SpeechRecognition processes and generates text -> Sent to backend via JSON) OR (Phone Mode: Twilio streams mu-law bytes -> WebSocket receives -> Silence detected -> Wrapped in WAV -> Uploaded to Groq Whisper -> Text response returned -> Appended to session memory).

DEEPGRAM STARTUP WARNING SIGN

When the backend server starts up, it logs status indicators for each service:

Deepgram (speech-to-text) -> not configured

It also shows warning log: DEEPGRAM_API_KEY not set.

- Why does it show this?

The current version of the project is fully integrated with Groq's Whisper API (whisper-large-v3-turbo) for speech-to-text, which is fast and operates on Groq's free tier. Deepgram was originally planned as a streaming STT provider, but Groq's Whisper API was used instead to eliminate Deepgram subscription requirements. The warning is simply a cosmetic check for the optional DEEPGRAM_API_KEY in environment variables.

DEEPGRAM API KEY PURPOSE

In the project, the Deepgram API key is an optional variable. The backend contains placeholder configurations and comments demonstrating where a developer can swap out Groq's Whisper API for a Deepgram streaming connection if they require real-time, word-by-word transcription streaming for complex telephony deployments.

SECTION 6 - TEXT TO SPEECH

Text-to-Speech (TTS) converts the text generated by the AI Brain back into spoken audio, bringing Priya or Arjun to life.

WEB BROWSER TEXT-TO-SPEECH

The web browser mode uses a hybrid approach:

- Primary Engine: The client sends a request to the backend /api/tts route. The route calls the Microsoft Edge Neural TTS library (msedge-tts) to generate high-quality, natural-sounding audio streams which are sent back as MP3 blobs. The browser plays these blobs using HTML5 Audio.
- Fallback Engine: If the backend TTS route fails (e.g. timeout or server issue), the browser falls back to the local Web Speech API window.speechSynthesis. It creates a SpeechSynthesisUtterance and applies the best local voice matching en-IN or ta-IN.

TELEPHONY TEXT-TO-SPEECH

Twilio phone calls require a low-latency telephony-specific audio format (8kHz mono mu-law). Twilio cannot fetch Edge TTS audio directly.

- How it works:

1. The server calls the synthesizeSpeechMulaw function in src/services/tts.service.ts.
2. The function uses msedge-tts to generate the neural voice MP3 audio (e.g. en-IN-NeerjaNeural at 24kHz).
3. The MP3 buffer is fed directly into an FFmpeg stream (fluent-ffmpeg).
4. FFmpeg transcodes the audio: downsampling the frequency to 8000Hz, converting it to a single channel (mono), and applying pcm_mulaw encoding.
5. The resulting mu-law buffer is sent to Twilio over the WebSocket connection.

- Webhook Fallback voice:

If the WebSocket media stream fails, Twilio executes the fallback TwiML. The fallback TwiML utilizes Twilio's built-in text-to-speech engine inside the <Say> verb. This is configured to use Amazon Polly's Indian English female voice: <Say voice="Polly.Aditi" language="en-IN">.

EXACT NEURAL VOICE NAMES

The system utilizes Microsoft Edge's advanced neural voices:

- Priya (Female):

- * English / Tenglish: en-IN-NeerjaNeural (Edge Neural Voice, sounds like a warm Indian female advisor).
- * Tamil Script: ta-IN-PallaviNeural (Edge Neural Voice, speaks fluent Tamil).
- * Web Webhook Fallback: Polly.Aditi (Amazon Polly).

- Arjun (Male):

- * English / Tenglish: en-IN-PrabhatNeural (Edge Neural Voice, warm Indian male).
- * Tamil Script: ta-IN-ValluvarNeural (Edge Neural Voice, speaks fluent Tamil).
- * Web Webhook Fallback: Twilio basic male voice.

TAMIL VOICE INTEGRATION

In browser mode and telephony WebSocket streams, if the text contains Tamil Unicode script characters ([\u0B80-\u0BFF]), the TTS engine automatically switches to ta-IN-PallaviNeural (Priya) or ta-IN-ValluvarNeural

(Arjun). This allows the AI to speak fluent, natural Tamil with correct inflection. In Twilio fallback webhook mode, Twilio's Say verb does not support high-quality Tamil, so the fallback voice remains Indian English.

HOW TANGLISH WORKS WITH THE TTS ENGINE

Tanglish is Tamil words written using the English alphabet (e.g. "enna plan iruku?").

- Browser Fallback: The Web Speech API cannot read Tanglish natively; it would mispronounce words. The browser client features a phonetic mapper `tanglishToPhonetic()` which replaces Tanglish spellings with phonetic English approximations (e.g. "sari" -> "suh-ree", "romba" -> "rowm-ba") before sending text to the browser's TTS engine.
- Edge TTS Mode: The neural Indian English voices (`en-IN-NeerjaNeural` and `en-IN-PrabhatNeural`) have been trained on Indian English speech, which naturally includes Hinglish and Tanglish terms. These voices read Tanglish words natively with a natural Indian accent, eliminating the need for complex phonetic mapping.

THE /api/tts ROUTE BEHAVIOR

The Fastify route `/api/tts` (in `src/routes/tts.routes.ts`) handles browser synthesis requests:

1. Validates the request body (`text`, `agentName`, `languageMode`).
2. Cleans the text: strips markdown (bold, asterisks), removes debug brackets (like `[listening]`), and removes extra white spaces.
3. Automatically selects the voice: calls `selectVoiceForContent`. If the text contains any Tamil script characters, it selects `ta-IN-PallaviNeural/ta-IN-ValluvarNeural`. Otherwise, it selects `en-IN-NeerjaNeural/en-IN-PrabhatNeural`.
4. Instantiates `MsEdgeTTS`, requests the stream, collects audio chunks, and sends the compiled MP3 buffer back with `Content-Type: audio/mpeg`. If an error occurs, it returns `useBrowserFallback: true` to trigger browser-side TTS.

SECTION 7 - AI BRAIN

The AI Brain of Vizza Insure AI coordinates conversation flow, detects customer intent, and handles objections.

AI SERVICE AND MODEL DETAILS

The conversation engine is powered by Groq Cloud.

- Exact Model Name: llama-3.3-70b-versatile (configured in .env as GROQ_MODEL). If not specified, it falls back to llama3-8b-8192. Llama 3.3 70B is chosen for its high reasoning capabilities, fast response speeds, and ability to handle multilingual dialog.
- API Endpoint URL: The Groq SDK connects to Groq's official API endpoint: <https://api.groq.com/openai/v1/chat/completions>.

COMPLETE SYSTEM PROMPT

The system prompt is dynamically built inside buildSystemPrompt (in src/services/ai.service.ts) to adapt the AI's behavior based on the agent name and language mode:

You are {agentName}, a warm and fast voice assistant for Vizza Insurance.
Your CRITICAL rule is to match the user's language mode EXACTLY.

[PURE TAMIL MODE] (Active if customer speaks/writes in Tamil script)

- User writes in Tamil script. You MUST reply entirely in Tamil script.
- Write your full answer in Tamil.
- Use natural conversational Tamil, not formal written Tamil.
- Insurance terms like premium, policy, cover, claim can stay in English but write everything else in Tamil script.
- Example:
User: "Health insurance pathi sollu" (Tamil Script: "?????? ?????????? ????? ?????")
Agent: "Health Plus plan-il hospitalisation, surgery, daycare, ambulance ellam cover aagum. Kudumbathirku plan thevaiya?" (Tamil Script: "?????? ????? ?????????? ?????????????? ?????????, ????? ?????????, ?????????? ????????? cover ??????. ????? ?????????????????? plan ?????????")

[TANGLISH MODE] (Active if customer speaks Tanglish keywords like enna, sollu, sari, etc.)

- User speaks in Tanglish. Respond in Tanglish (Tamil words written in English letters).
- STRICT RULE: NEVER use Tamil script characters.
- Example:
User: "enna cover iruku health plan la"
Agent: "Health Plus la hospitalisation, surgery, medicines ellame cover aagum. Unga family ku plan venum-a?"

[ENGLISH MODE] (Active by default for English conversations)

- Respond in simple, warm Indian English.
- Example:
User: "what does health insurance cover"
Agent: "Health Plus covers hospitalisation, surgery, daycare procedures and ambulance charges. Would you like to know about the premium?"

[VIZZA INSURANCE PRODUCTS]

- Vizza Health Plus: hospitalisation, 3L-1Cr, from 599/month.
- Vizza Drive Safe: Motor insurance, claims in 7 days.
- Vizza Life Secure: Term life up to 5Cr, from 500/month.
- Vizza Home Guard: Home insurance, from 2000/year.

[STRICT RULES]

- Max 3 sentences per response (4 sentences for Tamil mode).
- End with a short question.
- Match the user's detected mode exactly. Never mix modes.
- Plain text only (no markdown, no bold, no lists).

CONVERSATION HISTORY MAINTENANCE

- Web Browser Conversations:

Stored client-side in the conversationHistory array. When a user sends a message, it is pushed to the array. The array is sent to the backend /test/chat route, which forwards it to Groq.

- Telephony WebSocket Conversations:

Stored in the backend sessionManager (in src/services/session.service.ts). Each active call session contains a conversationHistory array of message objects.

- Fallback Webhook Telephony:

Stored in callSessions Map inside src/routes/call.routes.ts. It maps CallSid to history arrays.

LANGUAGE DETECTION MECHANISM

Language detection is performed by detectLanguage() (in src/services/ai.service.ts):

1. It checks the text for Tamil Unicode characters ([\u0B80-\u0BFF]). If found, returns tamil.
2. It checks for common Tanglish keywords (enna, sollu, sari, illa, iruku, romba, nalla, konjam, yevlo, evlo, venduma, vendam, epdi, namba, machi, vanakkam, naan, unga).
3. If the message matches any keyword or unicode range, the function returns tamil. Otherwise, it returns english.

DYNAMIC SYSTEM PROMPT SHIFTING

When the customer submits an utterance, the server runs detectLanguage(). The result is passed to buildSystemPrompt(). If tamil is detected, the prompt checks if the input has Tamil Unicode script. If yes, it sets PURE TAMIL MODE. If no, it sets TANGLISH MODE. Otherwise, it sets ENGLISH MODE. This ensures the prompt changes dynamically after each customer turn.

RESPONSE TOKEN LIMITS

- English Responses: max_tokens is set to 150.
- Tamil / Tanglish Responses: max_tokens is set to 300 (Tamil characters require higher token allocations).
- The completion call also includes stop sequences [".", "?", "!", "?"] to force the model to end on sentence boundaries, keeping replies concise.

FALLBACKS AND TIMEOUT HANDLING

To ensure the voice call does not hang in silence:

1. The server wraps the Groq API call in a Promise timeout set to 8 seconds (telephony route) or 2.5 seconds (WebSocket pipeline configuration).

2. If Groq API fails, times out, or has rate limit errors, the system catches the error and returns: `FALLBACK_RESPONSES.AI_ERROR` ("I apologize, I'm having a small technical issue. Let me connect you with one of our team members."). The call is then escalated to a human agent.

SECTION 8 - TELEPHONY LAYER

The telephony layer connects the Vizza Insure AI backend to physical mobile networks, turning the AI server into a phone agent.

TELEPHONY SERVICE: TWILIO

The system integrates with Twilio, a cloud communications platform, to route and manage phone calls.

CONFIGURED TWILIO NUMBER

The project utilizes the verified Twilio phone number: +1 814 885 1742 (configured in .env as TWILIO_PHONE_NUMBER).

OUTBOUND CALL FLOW

1. An admin clicks "Call Now" on the dashboard, triggering an HTTP POST to /api/call/start.
2. The server receives the request, validates the parameters, and invokes the Twilio client SDK:

```
`const call = await twilioClient.calls.create({
  to: phone,
  from: env.TWILIO_PHONE_NUMBER,
  url:
    `\\${env.BASE_URL}/api/call/answer?agentName=${agentName}&customerName=${encodeURIComponent(customerName)}`,
  statusCallback: `\\${env.BASE_URL}/api/call/status`,
  method: 'POST'
});`
```

3. Twilio initiates the call over the public telecommunication network.
4. The customer's physical phone rings.

WHAT HAPPENS WHEN THE CUSTOMER ANSWERS

When the customer answers the call, Twilio sends an HTTP POST request to the Answer Webhook URL (/api/call/answer).

- Webhook Action:

The server responds with Twilio Markup Language (Twiml) instructing Twilio to establish a WebSocket stream:

```
`<Response>
  <Connect>
    <Stream url="wss://saggy-aloof-attendee.ngrok-free.dev/api/calls/media-stream">
      <Parameter name="agentName" value="Priya" />
      <Parameter name="customerName" value="Customer" />
    </Stream>
  </Connect>
  <Say voice="Polly.Aditi" language="en-IN">Hello! I am Priya from Vizza Insurance. How can I help you today?</Say>
```

```
                                <Gather                                input="speech"
action="https://saggy-aloof-attendee.ngrok-free.dev/api/call/respond?agentName=Priya"    method="POST"
```

```
language="en-IN" speechTimeout="3" speechModel="phone_call"/>
</Response>
```

WHAT HAPPENS WHEN THE CUSTOMER SPEAKS

The audio processing depends on the active connection mode:

- WebSocket Stream Mode (Primary):

The customer's voice is streamed in real-time. The server detects silence, transcribes the voice using Groq Whisper, queries Groq AI, synthesizes the audio via Edge TTS, and streams the mu-law chunks back to Twilio.

- Webhook Fallback Mode:

If the WebSocket fails, Twilio falls back to the <Gather> verb. Twilio transcribes the speech on its own servers and sends an HTTP POST to /api/call/respond containing the SpeechResult parameter.

1. The server receives the text in request.body.SpeechResult.
2. Queries Groq AI with the text and conversation history.
3. Returns a TwiML response containing the AI's answer inside a <Say> verb, followed by another <Gather> verb to keep the call loop going.

CONVERSATION LOOP MECHANICS

The conversation loop is maintained dynamically:

- In WebSocket mode: The WebSocket connection remains open. The loop is managed in the server code: capturing audio, checking energy levels, and streaming audio responses without requiring webhook restarts.
- In Webhook mode: The loop is maintained by returning TwiML containing <Say> followed by <Gather>. When the customer replies, Twilio calls /api/call/respond again, starting the loop over.

WHAT HAPPENS WHEN THE CALL ENDS

When the call is hung up, Twilio sends an HTTP POST callback to the Status Webhook (/api/call/status).

- Action: The server receives the status (e.g. completed or failed) and call duration. It removes the active session from the sessionManager Map and logs the call outcome to the SQLite database.

TWILIO CONSOLE WEBHOOK URL CONFIGURATION

To route incoming calls or process outbound responses correctly, developers must configure the following Webhook URL in their Twilio Console under the active phone number's "A Call Comes In" section:

- Webhook URL: <https://saggy-alooof-attendeo.ngrok-free.dev/api/call/answer> (configured as HTTP POST).

TWILIO VOICES FOR INDIAN ENGLISH

For built-in TwiML synthesis (fallback mode), the server uses Amazon Polly's high-quality neural voice Polly.Aditi with the language set to en-IN (Indian English). This provides a natural, accent-compatible voice for Indian users.

TAMIL LANGUAGE SUPPORT ON TELEPHONY CALLS

- WebSocket Mode: Fully supported. The server synthesizes Tamil text using Microsoft Edge's Tamil neural voice (ta-IN-PallaviNeural) and streams the transcoded mu-law audio to Twilio.
- Fallback Webhook Mode: Limited. Because Twilio's standard <Say> verb does not support high-quality Tamil neural voices natively, the system will fall back to speaking English.

CONVERSATION MEMORY STORAGE

Telephony memory is stored in-memory on the backend:

- WebSocket Mode: Stored in the sessionManager instance.
 - Webhook Mode: Stored in the callSessions Map inside call.routes.ts.
- Both map the conversation history array to the unique Twilio CallSid.

SECTION 9 - DATABASE

Vizza Insure AI features a fully functional database layer used to log call parameters, track outreach campaigns, and save generated sales leads.

DATABASE CONNECTION AND STATUS

- Status: The database is connected and active.
- Technology: SQLite is used as the relational database engine.
- Connection: Managed through Prisma ORM. The connection string is defined in .env as `DATABASE_URL="file:./dev.db"`. In production, this can be swapped to PostgreSQL (e.g. Supabase) by changing the provider in `schema.prisma` and updating the database URL.

DATABASE TABLE SCHEMAS

The SQLite database contains five tables managed via Prisma:

1. Table: customers (mapped from Customer model)

Stores customer profiles and preferences.

- id (String, Primary Key): Unique CUID identifier.
- name (String): Customer name.
- phone (String, Unique): Customer phone number.
- email (String, Optional): Email address.
- preferredCallStart (String, Optional): Start of preferred calling hours (HH:mm format).
- preferredCallEnd (String, Optional): End of preferred calling hours (HH:mm format).
- timezone (String): Default "Asia/Kolkata".
- doNotCall (Boolean): Default false. If true, customer will not be dialed.
- lastCalledAt (DateTime, Optional): Timestamp of last call.
- isDeleted (Boolean): Logical deletion flag.
- createdAt / updatedAt (DateTime): Auto-generated timestamps.

2. Table: call_logs (mapped from CallLog model)

Records the details of every outbound phone call.

- id (String, Primary Key): Unique CUID identifier.
- callSid (String, Unique): Twilio Call SID.
- customerId (String, Optional, Foreign Key): Links to customers.id.
- customerPhone (String): Logged phone number.
- customerName (String, Optional): Logged name.
- status (String): Call status (e.g., INITIATED, IN_PROGRESS, COMPLETED, FAILED).
- duration (Int, Optional): Call duration in seconds.
- conversation (String, Optional): JSON string containing the full conversation transcript.
- escalated (Boolean): Default false. Set to true if transferred to a human.
- escalationReason (String, Optional): Reason for escalation.
- callState (String): Active pipeline state (e.g., GREETING, DISCOVERY).
- retryCount (Int): Default 0. Tracks dialing retry attempts.

- campaignId (String, Optional, Foreign Key): Links to campaigns.id.
- createdAt / updatedAt (DateTime): Auto-generated timestamps.

3. Table: leads (mapped from Lead model)

Registers sales opportunities extracted from successful calls.

- id (String, Primary Key): Unique CUID.
- phone (String): Customer phone number.
- name (String, Optional): Customer name.
- interestedIn (String, Optional): Policy type interested in (HEALTH, MOTOR, LIFE, HOME).
- callLogId (String, Optional, Foreign Key): Links to call_logs.id.
- customerId (String, Optional, Foreign Key): Links to customers.id.
- objections (String, Optional): Logged customer objections.
- status (String): Lead status (e.g., NEW, CONTACTED, FOLLOW_UP, CONVERTED, LOST).
- notes (String, Optional): Sales notes.
- createdAt / updatedAt (DateTime): Auto-generated timestamps.

4. Table: campaigns (mapped from Campaign model)

Outbound campaign outreach logs.

- id (String, Primary Key): Unique CUID.
- name (String): Campaign description.
- status (String): Default "ACTIVE".
- totalCustomers (Int): Total target count.
- callsMade (Int): Number of calls placed.
- answeredCount (Int): Number of answered calls.
- createdAt / updatedAt (DateTime): Auto-generated timestamps.

5. Table: call_events (mapped from CallEvent model)

Granular logging of pipeline execution steps.

- id (String, Primary Key): Unique CUID.
- callSid (String, Foreign Key): Links to call_logs.callSid.
- eventType (String): Event description (e.g., utterance_received, ai_response_sent).
- payload (String, Optional): JSON string containing logs metadata.
- createdAt (DateTime): Auto-generated timestamp.

DATA SAVED AFTER PHONE CALLS

When a campaign phone call concludes:

1. The status webhook updates the call_logs table, changing the status column to "COMPLETED" or "FAILED" and saving the call duration.
2. The server serializes the in-memory conversation history array into a JSON string and updates the conversation column in call_logs.
3. If an escalation occurred, the escalated and escalationReason columns are updated.
4. Granular CallEvent rows remain in the database for post-call audit trails.

DATA SAVED AFTER WEB CONVERSATIONS

In the current version of the project, web-based chat and voice conversations are not written to the SQLite database. They are maintained entirely in client-side memory. This keeps the testing environment fast and avoids database bloat during developer testing.

HOW FRONTEND DASHBOARD ACCESSES DATABASE

The frontend dashboard reads database records using HTTP GET requests. The request must include the authentication API key inside the headers:

```
`const response = await fetch('/dashboard/analytics', {  
  headers: { 'x-api-key': 'dev-secret-key' }  
});`
```

The server queries the SQLite database using Prisma, aggregates the records, and returns the compiled data.

DISCONNECTED DATABASE FALLBACK

If the database connection fails (e.g., DATABASE_URL is empty or the database file is corrupted):

1. The backend server logs a warning: DATABASE_URL not configured - running without database. Data will NOT be persisted.
2. The server continues to start up.
3. Conversations (both web and phone calls) still work, since active sessions are stored in-memory inside the Fastify maps.
4. However, call histories, transcripts, events, and leads will not be saved to disk. When the Node.js server restarts, all historical logs will be lost.

SECTION 10 - HOW TO RUN THE PROJECT

Follow this step-by-step setup guide to run Vizza Insure AI on a Windows computer.

STEP 1: INSTALL NODE.JS

1. Download Node.js version 20.x or higher from the official website (<https://nodejs.org>).
2. Run the installer, accept the terms, and click install.
3. Verify the installation by opening PowerShell and running:

```
`node --version`  
`npm --version`
```

STEP 2: DOWNLOAD THE PROJECT FILES

1. Extract the project files to a local directory (e.g., C:\Users\god\Documents\Voice assistant).
2. Open PowerShell and navigate to the project directory:

```
`cd "C:\Users\god\Documents\Voice assistant"`
```

STEP 3: RUN NPM INSTALL

1. Run the installation command:

```
`npm install`
```

This command reads package.json and downloads the required dependencies (Fastify, Prisma, Twilio, Edge-TTS, FFMpeg, etc.) into the node_modules folder.

STEP 4: SET UP ENVIRONMENT VARIABLES

1. Duplicate .env.example and name the new file .env.
2. Open .env in a text editor and configure the variables:
 - PORT: Set to 3000.
 - DATABASE_URL: Set to "file:./dev.db".
 - GROQ_API_KEY: Enter your Groq API key (starts with gsk_). You can generate one at <https://console.groq.com>.
 - TWILIO_ACCOUNT_SID: Enter your Twilio Account SID (found on your Twilio console dashboard).
 - TWILIO_AUTH_TOKEN: Enter your Twilio Auth Token.
 - TWILIO_PHONE_NUMBER: Enter your Twilio virtual phone number.
 - BASE_URL: Set to <https://saggy-aloof-attendee.ngrok-free.dev>

STEP 5: SET UP NGROK

Because Twilio needs a public URL to send webhook notifications, you must open a tunnel using ngrok:

1. Download ngrok from the official website (<https://ngrok.com>) and install it.
2. Open a new PowerShell terminal and run:

```
`ngrok http 3000`
```
3. Copy the public forwarding HTTPS URL (e.g., <https://saggy-aloof-attendee.ngrok-free.dev>).
4. Paste this URL into your .env file as the BASE_URL value.

STEP 6: START THE BACKEND SERVER

1. Initialize the Prisma database:

```
`npx prisma db push`
```

2. Start the server in development mode:

```
`npm run dev`
```

The console will display a block showing that the Vizza Insure AI server is running and which services are active.

STEP 7: OPEN THE FRONTEND IN A BROWSER

1. Open Google Chrome or Microsoft Edge.
2. Navigate to: <http://localhost:3000/browser-call.html> (or the ngrok HTTPS equivalent).
3. If using browser mode, click "Start Call" to test the voice agent. Ensure you grant microphone access when prompted.

STEP 8: CONFIGURE TWILIO WEBHOOKS

1. Log in to the Twilio Console (<https://console.twilio.com>).
2. Go to Phone Numbers -> Manage -> Active Numbers.
3. Click on your active Twilio number (+1 814 885 1742).
4. Scroll down to the "Voice & Fax" configuration block.
5. Under "A Call Comes In", select "Webhook" and paste your ngrok URL with the answer suffix:

```
`https://saggy-aloof-attendeo.ngrok-free.dev/api/call/answer`
```
6. Set the method to HTTP POST and click Save.

STEP 9: PLACE A TEST CALL

1. Open the browser call dashboard.
2. Scroll to the "Outbound Telephony" section.
3. Input your verified personal phone number, enter your name, select Priya or Arjun, and click "Call Now".
4. Answer your phone when it rings. Speak to the agent to verify the pipeline is working.

SECTION 11 - COMPLETE LIST OF ALL API KEYS AND SERVICES

Service Name	Purpose in Project	Sign-up Location	Where Stored in Project	Free Tier Limit
Twilio	Initiates outbound calls, routes incoming webhooks, and streams	https://twilio.com	Stored in .env as TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER.	\$15 trial credit for calls.
Groq Cloud	Processes audio.speech-to-text (Whisper Large V3) and AI response generation (Llama3) speech-to-text	https://console.groq.com	Stored in .env as GROQ_API_KEY and GROQ_ENDPOINT.	Generous free limits (e.g. 14,400 requests/day).
Deepgram	Optional (training) speech-to-text integration (currently inactive).	https://deepgram.com	Stored in .env as DEEPGRAM_API_KEY.	\$200 free credit.
Supabase	Optional database service (PostgreSQL). Can replace the local database.	https://supabase.com	Stored in .env as DATABASE_URL.	500MB free PostgreSQL database.
ngrok	Creates a secure HTTPS tunnel to route Twilio calls to localhost.	https://ngrok.com	Stored in .env as BASE_URL.	1 free static agent tunnel.
Microsoft Edge TTS	Synthesizes natural-sounding voice audio in real-time.	Built-in API integration	Utilized in routes/tts.routes.ts and services/tts.service.ts.	100% Free with no API keys required.

SECTION 12 - KNOWN ISSUES AND LIMITATIONS

Below is a list of known issues and technical limitations in the current version of the project:

1. Browser Audio Autoplay Restrictions

- Issue: Web browsers block audio playback until the user interacts with the page.
- Trigger: Occurs if the page tries to play the agent's greeting before the user clicks a button.
- Fix: The page uses an "unlock audio" mechanism. The user must click "Start Call" before the agent plays the audio.

2. Deepgram Warning Symbol on Startup

- Issue: The terminal displays a red warning symbol next to Deepgram on server startup.
- Trigger: Occurs because the server checks for DEEPGRAM_API_KEY, which is empty by default.
- Fix: This is a cosmetic warning. The system uses Groq Whisper for speech-to-text instead of Deepgram. Developers can ignore this warning or add a dummy key to hide it.

3. Call Session Memory Leak Risks

- Issue: Active conversation histories are stored in-memory inside the sessions Map. If a call is interrupted and the status webhook fails to deliver, the session remains in memory.
- Trigger: Occurs during network drops or forced server shutdowns.
- Fix: Implement an automatic session cleanup job that sweeps and removes sessions older than 30 minutes.

4. SQLite Concurrent Access Limits

- Issue: SQLite locks the database file when writing data. Under heavy call loads (e.g. massive campaigns), concurrent writes can fail.
- Trigger: Occurs during high-volume campaign calls.
- Fix: Update the database to a PostgreSQL instance (e.g. Supabase) by changing the provider in schema.prisma and updating the database URL.

5. Telephony Fallback Mode Voice Restrictions

- Issue: Telephony fallback mode uses Twilio's <Say> verb, which does not support high-quality Tamil neural voices.
- Trigger: Occurs if the WebSocket media stream drops and the system falls back to webhook mode.
- Fix: Ensure the WebSocket connection is stable, or use a custom Twilio TTS integration that fetches Edge TTS audio URLs directly.

SECTION 13 - FUTURE IMPROVEMENTS

The following features are planned for future versions of the project:

1. FreeSWITCH Telephony Integration

- Goal: Replace Twilio with FreeSWITCH (an open-source telephony platform).
- Benefit: Bypasses Twilio's per-minute calling charges by routing calls through custom SIP trunks, reducing operational costs to near zero.

2. CSV Customer Upload for Calling Campaigns

- Goal: Build a bulk upload interface directly into the campaign manager.
- Benefit: Allows the operations team to upload customer lists using CSV files instead of pasting comma-separated numbers manually.

3. Supabase Analytics Dashboard

- Goal: Build a real-time call analytics dashboard powered by Supabase.
- Benefit: Provides visual charts showing call answer rates, lead conversions, agent performance, and campaign metrics.

4. Dynamic Lead Conversion Workflows

- Goal: Automatically sync converted leads with sales CRMs (e.g. Salesforce or HubSpot).
- Benefit: Ensures sales agents can follow up on leads generated by the AI immediately.